

BE Major Project

Project Summary & Claude Handoff

Working project brief • consolidated from project decisions and uploaded research notes

1. Executive Summary

The project is focused on multilingual AI safety for Indian/Indic languages, with the dataset generator as the highest-priority engineering component. The central research problem is that safety behavior demonstrated in English cannot be assumed to transfer reliably to Indian-language, romanized, code-mixed, multilingual, or adversarial prompts. The intended system should therefore generate a high-quality, structured safety/jailbreak dataset and use it to support training and rigorous evaluation of a safety classifier/guard model.

The uploaded literature notes identify IndicGuard, IndicJR, Languages Change, Safety Weakens, and related jailbreak/safety benchmarks as especially close prior work. Llama Guard is identified as a standard safety baseline, while MuRIL, IndicBERT/IndicBERT v2, XLM-R, and mBERT are candidate model families for classifier implementation. These literature points are source-derived and should be verified against the original papers before being used as final claims.

2. Project Objectives & Requirements

- **Primary objective:** Build an Indic-language AI-safety dataset generation and evaluation pipeline, with the dataset generator treated as the top implementation priority.
- **Research objective:** Study whether and how safety/jailbreak robustness changes across Indian/Indic languages and across multilingual, romanized, code-mixed, and adversarial prompt forms.
- **Model objective:** Develop/evaluate a safety classification or guard-model component that can consume the generated dataset and classify unsafe/safe content and/or attack types, depending on the final experimental design.
- **Evaluation objective:** Benchmark the proposed approach against relevant existing models/benchmarks rather than evaluating it in isolation.
- **Engineering objective:** Create a reproducible pipeline: data generation → validation/filtering → dataset formatting → train/validation/test split → model training → benchmark evaluation → error analysis.

3. Scope & Dataset Requirements

- Indic/Indian-language safety data should be central to the dataset rather than treating English as the only source language.
- Include adversarial/jailbreak prompts because jailbreak robustness is a major part of the stated research problem.
- Account for Romanized text and code-mixed prompts; the uploaded notes specifically identify these as important dimensions in IndicJR and related work.
- Support multilingual and mixed-language variations, including translated or structurally modified prompts where appropriate.

- Include clear labels and metadata so experiments can be sliced by language, script, prompt type, attack type, safety category, and generation source.
- Maintain held-out evaluation data and avoid leakage between generated variants and test examples.
- Use a validation layer to identify duplicates, malformed examples, ambiguous labels, unsafe artifacts, and low-quality generated samples.
- Keep a provenance record for each example: generator/model, prompt template or transformation, language, attack family, timestamp/version, and validation status.

4. Technical Architecture

Recommended end-to-end architecture (architecture is a working implementation plan; it is not presented as already completed):

1. Configuration layer — languages, safety categories, attack families, generation counts, model/provider settings, seeds, and experiment versions.
2. Prompt/template engine — deterministic templates plus controlled adversarial transformations for translation, romanization, code-mixing, jailbreak framing, structural variation, and related attack families.
3. Dataset generator — calls the selected generation model(s), records raw outputs, and stores complete provenance.
4. Validation/filtering layer — schema validation, duplicate detection, quality checks, language/script checks, label consistency checks, and optional safety review.
5. Dataset builder — converts validated records into canonical JSONL/CSV/Parquet or the selected final format, with reproducible train/validation/test splits.
6. Classifier/guard model — fine-tune or evaluate a multilingual encoder/guard architecture against the generated data.
7. Benchmark adapter — evaluates on external benchmarks such as IndicJR and other relevant datasets where licensing/access permits.
8. Metrics + analysis — accuracy/F1 where classification is appropriate, attack success rate/jailbreak robustness for attack evaluation, per-language breakdowns, confusion matrices, and error analysis.
9. Experiment tracking — save configurations, model versions, dataset versions, metrics, and artifacts so results are reproducible.

5. Candidate Tools & Models

Component	Candidates / references	Status
Safety baseline	Llama Guard	Literature-identified baseline; verify exact version/configuration
Indic encoders	MuRIL; IndicBERT; AI4Bharat IndicBERT v2	Candidate model families
Multilingual baselines	XLM-RoBERTa; mBERT	Candidate model families
Jailbreak evaluation	IndicJR; Indic-JailbreakBench / related benchmarks	Literature-identified; verify availability/licensing
Hallucination evaluation	BHRAM-IL; MultiHaluDet	Relevant only if hallucination detection remains in scope
Adversarial methodology	JailGuard; Mātṛkā	Methodology references

6. Decisions Already Made

- The dataset generator is the top priority of the project.

- The project is fundamentally centered on multilingual/Indic AI safety rather than English-only safety.
- Jailbreak robustness is a core research dimension.
- Romanized and code-mixed inputs are important rather than optional edge cases.
- The project should compare against existing models/benchmarks instead of proposing an isolated system.
- Llama Guard is a key safety baseline identified in the research notes.
- MuRIL, IndicBERT/IndicBERT v2, XLM-R, and mBERT are candidate implementation baselines.
- Hallucination detection is a possible extension/related track, but the uploaded notes do not establish that it is a mandatory core deliverable. Do not let hallucination work displace the dataset-generator priority unless the project team explicitly decides otherwise.

7. What the Project Should Ultimately Produce

The working interpretation is a multi-part research/engineering deliverable rather than a single model:

- A reproducible Indic safety/jailbreak dataset generator.
- A curated, validated dataset with rich metadata and documented generation methodology.
- A trained/evaluated safety classifier or guard-model component, if the final implementation confirms this as the model deliverable.
- A benchmark/evaluation pipeline comparing the proposed system with established baselines and relevant Indic benchmarks.
- A safety evaluation framework/report containing per-language, per-attack, and aggregate results.

Important: the uploaded source does not by itself establish a final single 'output type' or exact model architecture. Claude should preserve this distinction and ask before locking an implementation detail that the team has not explicitly decided.

8. Completed vs Pending

Area	Current state	Next action
Literature scan	Initial relevant-paper set identified	Verify original papers and extract exact datasets, languages, metrics, licenses, and baselines
Problem direction	Indic multilingual safety/jailbreak direction established	Freeze a precise research question and hypotheses
Priority	Dataset generator explicitly prioritized	Implement generator MVP first
Model shortlist	Candidate model families identified	Run a small baseline comparison before committing
Benchmark shortlist	IndicJR and related work identified	Confirm access, task definition, and evaluation protocol
Dataset schema	Requirements inferred; final schema not supplied	Define canonical schema before large-scale generation
Generator implementation	Not established as completed in the supplied materials	Build, test, version, and benchmark the generator
Classifier training	Not established as completed	Train only after dataset quality gate
Final evaluation	Pending	Run held-out + external benchmark evaluation
Documentation	This handoff document created	Maintain it as decisions and code mature

9. Important Files & Code

Files available/identified in the supplied project context:

- **research papers.pdf** — Uploaded research-paper/related-work notes. It identifies the closest papers, benchmarks, model candidates, and research relevance.
- **Project conversation context** — Contains prior discussion that the dataset generator is the top priority and that the project needs clarity on whether the final solution is a dataset, classifier, safety framework, or combination.

No verified source-code repository, module names, API keys, environment files, dataset files, or finalized schema were present in the supplied source material for this summary. Claude must not invent filenames, functions, datasets, credentials, or completed implementation status. When code is provided later, update this section with exact paths and entry points.

10. Research References Identified in the Uploaded Notes

- IndicGuard: A Multilingual Safety Guard Model and Dataset for Indic Languages (2026) — identified as especially close to the project.
- IndicJR: A Judge-Free Benchmark of Jailbreak Robustness in South Asian Languages (EACL 2026) — identified as a strong benchmark reference, including 12 Indic languages, Romanized text, code-mixed prompts, and attack success rates.
- Languages Change, Safety Weakens: Jailbreaking LLMs Across English and Indian Languages (2026) — identified as directly aligned with the research problem.
- BHRAM-IL: Benchmark for Hallucination Recognition in Multiple Indian Languages (2025) — relevant if hallucination detection is included.
- Llama Guard: LLM-based Input-Output Safeguard (Meta, 2023) — identified as a standard safety baseline.
- JailGuard: Universal Detection Framework for Prompt-based Attacks (2025) — relevant to prompt injection/jailbreak detection methodology.
- Māṭṛkā: Multilingual Jailbreak Evaluation of Open-Source LLMs — relevant to mixed-language, translated, homoglyph, and structural jailbreak strategies.
- Detection and Analysis of Prompt Injection in Indian Multilingual LLMs — relevant to prompt injection in Indian multilingual models.
- MultiHaluDet — relevant to multilingual hallucination detection and cross-language transfer.
- SEALGuard — relevant supporting work on multilingual safety guard models.

11. Exact Instructions for Claude

Use the following as the operating brief when asking Claude to help with this project:

10. Treat the dataset generator as the highest-priority component. Do not jump straight to model fine-tuning before the data pipeline is designed and tested.
11. Read the project's research notes first. Preserve the terminology and research framing from the supplied literature, and verify original papers before making precise claims.
12. Do not invent missing project decisions. If the exact languages, labels, attack taxonomy, model, API/provider, dataset size, or benchmark protocol is not specified, propose concrete options and ask for confirmation before hard-coding them.

13. Build the project modularly: configuration → generation → validation → dataset construction → training/evaluation → reporting.
14. Make every generated example traceable through metadata. Reproducibility and versioning are requirements, not optional extras.
15. Design the dataset to support per-language and per-attack analysis. Do not collapse all Indic languages into one undifferentiated category.
16. Include romanized and code-mixed variants as first-class dataset dimensions.
17. Keep raw generated data separate from validated/curated data. Never overwrite raw generations.
18. Create deterministic seeds and configuration files wherever practical so experiments can be reproduced.
19. Add tests for schema validity, duplicates, language/script metadata, label validity, and split leakage before scaling generation.
20. When evaluating, report aggregate metrics plus language-wise and attack-wise breakdowns. Avoid relying on one aggregate score.
21. Use external benchmarks only according to their actual task definitions and access/licensing constraints. Do not claim comparability when task definitions differ.
22. If hallucination detection is explored, keep it as a clearly separated extension unless the team explicitly promotes it to a core objective.
23. Keep code readable and research-grade: clear modules, typed/configured interfaces where appropriate, logging, error handling, checkpoints, and README instructions.
24. Before declaring a milestone complete, show the exact files created/changed, how to run them, expected inputs/outputs, and a small validation result.

12. Recommended Immediate Execution Order

25. Freeze the one-sentence research question and the core deliverables.
26. Freeze the initial language list and safety/attack taxonomy.
27. Define the canonical dataset schema and metadata contract.
28. Implement a small dataset-generator MVP with a few languages and attack categories.
29. Add validation, deduplication, provenance tracking, and reproducible splits.
30. Generate a pilot dataset and manually inspect a representative sample.
31. Measure dataset quality before large-scale generation.
32. Implement baseline classifier/guard experiments using the selected candidate models.
33. Evaluate against relevant external benchmarks and baselines.
34. Perform per-language/per-attack error analysis and iterate on the generator.
35. Scale the dataset only after the pilot passes the quality gate.
36. Package the final dataset, code, experiment configurations, results, and documentation for the project report.

13. Source Note & Evidence Boundary

This summary is grounded in the uploaded file 'research papers.pdf' and the project decisions visible in the current shared-project context. The uploaded file is a four-page literature-notes document rather than the full text of all cited papers. Accordingly, claims about the cited papers are retained at the level supported by those notes; exact benchmark protocols, dataset sizes, model configurations, licenses, and quantitative results should be checked against the original publications before appearing in the final report.

Source citation: uploaded research-paper notes. [filecite](#) [turn0file0](#)